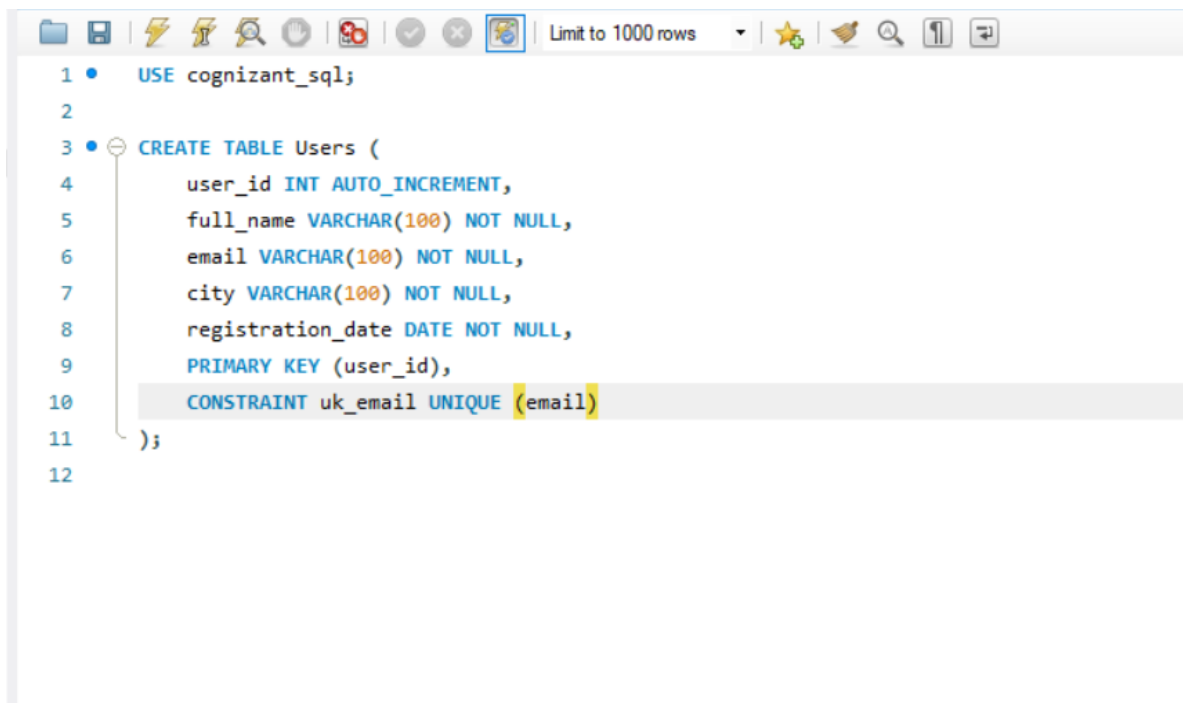


ANSI SQL Using MySQL Exercises

Database Schema

1. Users

Column Name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	User's email
city	VARCHAR(100)	NOT NULL	City of the user
registration_date	DATE	NOT NULL	Date when user registered



```
1 • USE cognizant_sql;
2
3 • CREATE TABLE Users (
4     user_id INT AUTO_INCREMENT,
5     full_name VARCHAR(100) NOT NULL,
6     email VARCHAR(100) NOT NULL,
7     city VARCHAR(100) NOT NULL,
8     registration_date DATE NOT NULL,
9     PRIMARY KEY (user_id),
10    CONSTRAINT uk_email UNIQUE (email)
11 );
12
```

2. Events

Column Name	Data Type	Constraints	Description
event_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique event identifier
title	VARCHAR(200)	NOT NULL	Title of the event
description	TEXT		Event description

city	VARCHAR(100)	NOT NULL	City where event is held
start_date	DATETIME	NOT NULL	Start date and time of event
end_date	DATETIME	NOT NULL	End date and time of event
status	ENUM	('upcoming','completed','cancelled')	Current status of the event
organizer_id	INT	FOREIGN KEY → Users(user_id)	User who organized the event

```

1 • CREATE TABLE Events (
2     event_id INT AUTO_INCREMENT,
3     title VARCHAR(200) NOT NULL,
4     description TEXT,
5     city VARCHAR(100) NOT NULL,
6     start_date DATETIME NOT NULL,
7     end_date DATETIME NOT NULL,
8     status ENUM('upcoming', 'completed', 'cancelled'),
9     organizer_id INT,
10    PRIMARY KEY (event_id),
11    FOREIGN KEY (organizer_id) REFERENCES Users(user_id)
12 );

```

3. Sessions

Column Name	Data Type	Constraints	Description
session_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique session identifier
event_id	INT	FOREIGN KEY → Events(event_id)	Associated event
title	VARCHAR(200)	NOT NULL	Title of the session
speaker_name	VARCHAR(100)	NOT NULL	Name of speaker
start_time	DATETIME	NOT NULL	Session start time
end_time	DATETIME	NOT NULL	Session end time

```

1 • CREATE TABLE Sessions (
2     session_id INT AUTO_INCREMENT,
3     event_id INT,
4     title VARCHAR(200) NOT NULL,
5     speaker_name VARCHAR(100) NOT NULL,
6     start_time DATETIME NOT NULL,
7     end_time DATETIME NOT NULL,
8     PRIMARY KEY (session_id),
9     FOREIGN KEY (event_id) REFERENCES Events(event_id)
10 );

```

4. Registrations

Column Name	Data Type	Constraints	Description
registration_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique registration ID
user_id	INT	FOREIGN KEY → Users(user_id)	User who registered
event_id	INT	FOREIGN KEY → Events(event_id)	Registered event
registration_date	DATE	NOT NULL	Date of registration

```

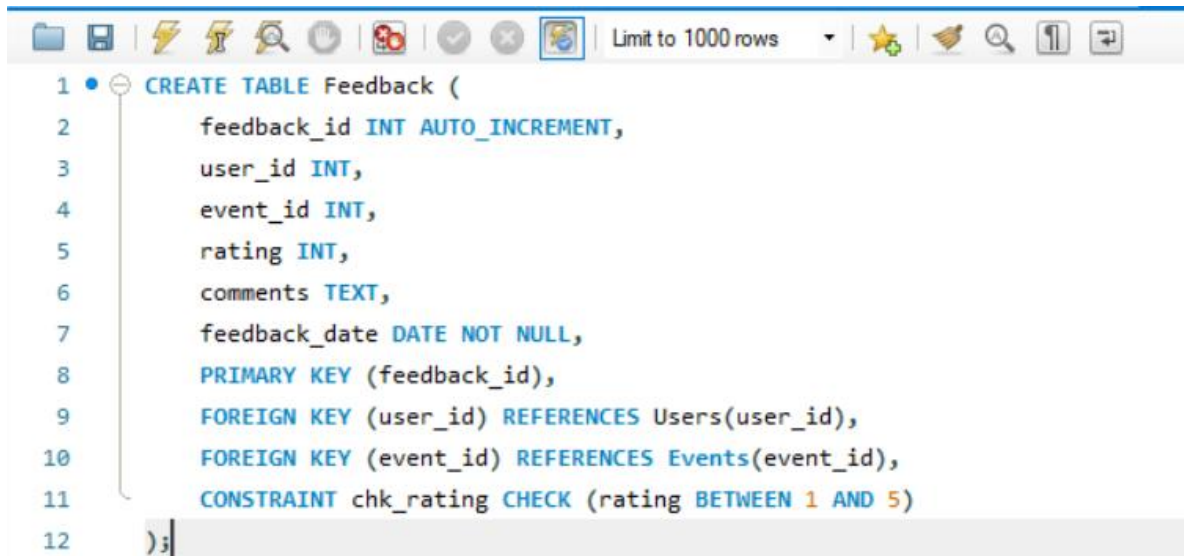
1 • CREATE TABLE Registrations (
2     registration_id INT AUTO_INCREMENT,
3     user_id INT,
4     event_id INT,
5     registration_date DATE NOT NULL,
6     PRIMARY KEY (registration_id),
7     FOREIGN KEY (user_id) REFERENCES Users(user_id),
8     FOREIGN KEY (event_id) REFERENCES Events(event_id)
9 );

```

5. Feedback

Column Name	Data Type	Constraints	Description
-------------	-----------	-------------	-------------

feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique feedback ID
user_id	INT	FOREIGN KEY → Users(user_id)	User who gave feedback
event_id	INT	FOREIGN KEY → Events(event_id)	Event for which feedback is given
rating	INT	CHECK (rating BETWEEN 1 AND 5)	Rating out of 5
comments	TEXT		Optional comments
feedback_date	DATE	NOT NULL	Date of feedback



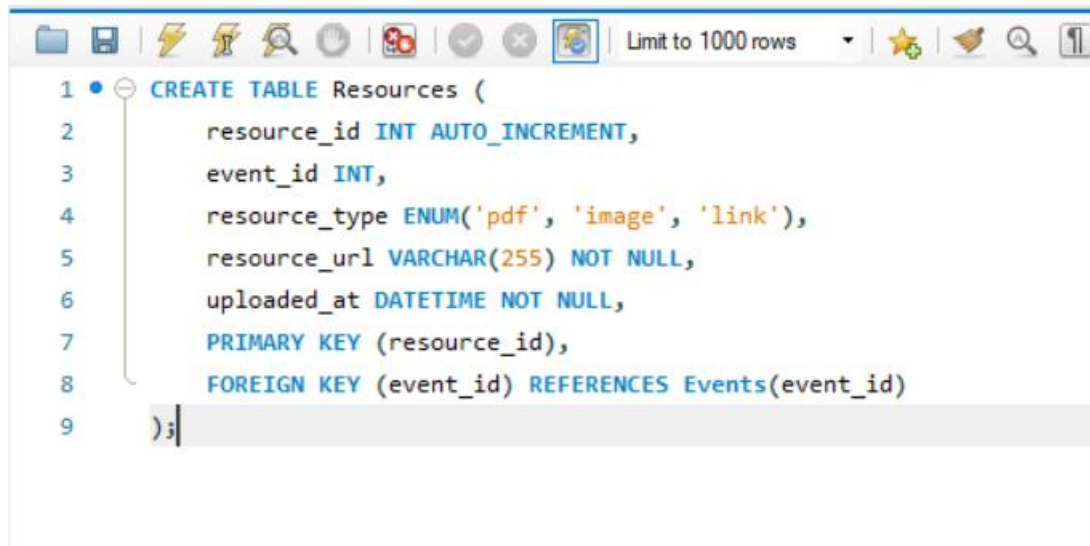
```

1 • CREATE TABLE Feedback (
2     feedback_id INT AUTO_INCREMENT,
3     user_id INT,
4     event_id INT,
5     rating INT,
6     comments TEXT,
7     feedback_date DATE NOT NULL,
8     PRIMARY KEY (feedback_id),
9     FOREIGN KEY (user_id) REFERENCES Users(user_id),
10    FOREIGN KEY (event_id) REFERENCES Events(event_id),
11    CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5)
12 );

```

6. Resources

Column Name	Data Type	Constraints	Description
resource_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique resource ID
event_id	INT	FOREIGN KEY → Events(event_id)	Associated event
resource_type	ENUM	('pdf','image','link')	Type of resource
resource_url	VARCHAR(255)	NOT NULL	Path or link to resource
uploaded_at	DATETIME	NOT NULL	Upload timestamp



The screenshot shows a SQL editor interface with a toolbar at the top containing icons for file operations, execution, and search. The main area displays a SQL script to create a table named 'Resources'. The script includes columns for 'resource_id' (auto-incrementing integer), 'event_id' (integer), 'resource_type' (enum with values 'pdf', 'image', 'link'), 'resource_url' (varchar(255), not null), and 'uploaded_at' (datetime, not null). It also defines a primary key on 'resource_id' and a foreign key on 'event_id' that references the 'Events' table. The script is numbered 1 through 9, and the cursor is positioned at the end of the final closing parenthesis and semicolon on line 9.

```
1 CREATE TABLE Resources (  
2     resource_id INT AUTO_INCREMENT,  
3     event_id INT,  
4     resource_type ENUM('pdf', 'image', 'link'),  
5     resource_url VARCHAR(255) NOT NULL,  
6     uploaded_at DATETIME NOT NULL,  
7     PRIMARY KEY (resource_id),  
8     FOREIGN KEY (event_id) REFERENCES Events(event_id)  
9 ) ;
```

Sample Dataset

1. Users

user_id	full_name	email	city	registration_date
1	Alice Johnson	alice@example.com	New York	2024-12-01
2	Bob Smith	bob@example.com	Los Angeles	2024-12-05
3	Charlie Lee	charlie@example.com	Chicago	2024-12-10
4	Diana King	diana@example.com	New York	2025-01-15
5	Ethan Hunt	ethan@example.com	Los Angeles	2025-02-01

1	1	Opening Keynote	Dr. Tech	2025-06-10 10:00:00	2025-06-10 11:00:00
2	1	Future of Web Dev	Alice Johnson	2025-06-10 11:15:00	2025-06-10 12:30:00
3	2	AI in Healthcare	Charlie Lee	2025-05-15 09:30:00	2025-05-15 11:00:00
4	3	Intro to HTML5	Bob Smith	2025-07-01 10:00:00	2025-07-01 12:00:00

Query 1 cognizant_sql - Schema SQL File 3* SQL File 5* x SQL File 6* SQL File 7*

Limit to 1000 rows

1 • `SELECT * from sessions;`

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: `⌘A`

	session_id	event_id	title	speaker_name	start_time	end_time
▶	1	1	Opening Keynote	Dr. Tech	2025-06-10 10:00:00	2025-06-10 11:00:00
	2	1	Future of Web Dev	Alice Johnson	2025-06-10 11:15:00	2025-06-10 12:30:00
	3	2	AI in Healthcare	Charlie Lee	2025-05-15 09:30:00	2025-05-15 11:00:00
	4	3	Intro to HTML5	Bob Smith	2025-07-01 10:00:00	2025-07-01 12:00:00
★	NULL	NULL	NULL	NULL	NULL	NULL

4. Registrations

registration_id	user_id	event_id	registration_date
1	1	1	2025-05-01
2	2	1	2025-05-02
3	3	2	2025-04-30
4	4	2	2025-04-28
5	5	3	2025-06-15

Query 1 cognizant_sql - Schema SQL File 3* SQL File 5* x SQL File 6* SQL File 7*

Limit to 1000 rows

```
1 • SELECT * from registrations;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap C

	registration_id	user_id	event_id	registration_date
▶	1	1	1	2025-05-01
	2	2	1	2025-05-02
	3	3	2	2025-04-30
	4	4	2	2025-04-28
	5	5	3	2025-06-15
•	NULL	NULL	NULL	NULL

5. Feedback

feedback_id	user_id	event_id	rating	comments	feedback_date
1	3	2	4	Great insights!	2025-05-16
2	4	2	5	Very informative.	2025-05-16
3	2	1	3	Could be better.	2025-06-11

Query 1 cognizant_sql - Schema SQL File 3* **SQL File 5*** SQL File 6* SQL File 7* SQL File 8*

Limit to 1000 rows

```
1 • SELECT * from feedback;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell

	feedback_id	user_id	event_id	rating	comments	feedback_date
▶	1	3	2	4	Great insights!	2025-05-16
	2	4	2	5	Very informative.	2025-05-16
	3	2	1	3	Could be better.	2025-06-11
*	NULL	NULL	NULL	NULL	NULL	NULL

6. Resources

resource_id	event_id	resource_type	resource_url	uploaded_at
1	1	pdf	https://portal.com/resources/tech_meetup_agenda.pdf	2025-05-01 10:00:00
2	2	image	https://portal.com/resources/ai_poster.jpg	2025-04-20 09:00:00
3	3	link	https://portal.com/resources/html5_docs	2025-06-25 15:00:00

Query 1	cognizant_sql - Schema	SQL File 3*	SQL File 5* x	SQL File 6*	SQL File 7*	SQL File 8*
1 • <code>SELECT * from Resources;</code>						
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:						
	resource_id	event_id	resource_type	resource_url	uploaded_at	
▶	1	1	pdf	https://portal.com/resources/tech_meetup_age...	2025-05-01 10:00:00	
	2	2	image	https://portal.com/resources/ai_poster.jpg	2025-04-20 09:00:00	
	3	3	link	https://portal.com/resources/html5_docs	2025-06-25 15:00:00	
*	NULL	NULL	NULL	NULL	NULL	

Exercises

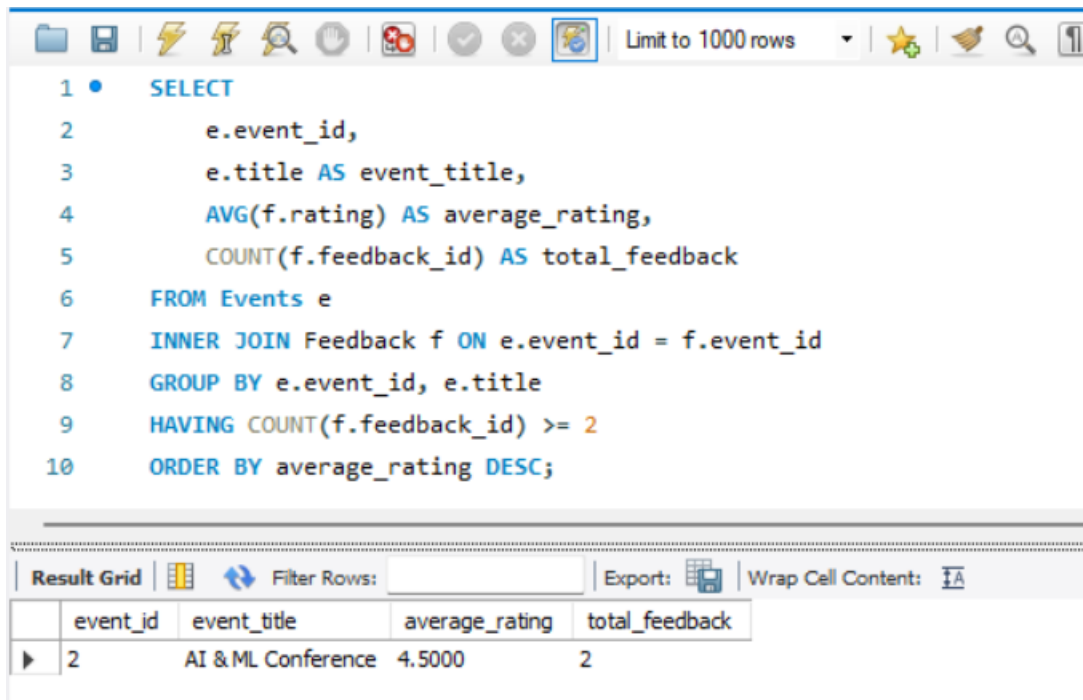
1. User Upcoming Events

Show a list of all upcoming events a user is registered for in their city, sorted by date.

<pre> 1 • SELECT 2 u.user_id, 3 u.full_name AS user_name, 4 u.city AS user_city, 5 e.event_id, 6 e.title AS event_title, 7 e.start_date, 8 e.status 9 FROM Users u 10 INNER JOIN Registrations r ON u.user_id = r.user_id 11 INNER JOIN Events e ON r.event_id = e.event_id 12 WHERE e.status = 'upcoming' 13 AND e.city = u.city 14 ORDER BY e.start_date ASC; </pre>							
Result Grid Filter Rows: Export: Wrap Cell Content:							
	user_id	user_name	user_city	event_id	event_title	start_date	status
▶	1	Alice Johnson	New York	1	Tech Innovators Meetup	2025-06-10 10:00:00	upcoming
	5	Ethan Hunt	Los Angeles	3	Frontend Development Bootcamp	2025-07-01 10:00:00	upcoming

2. Top Rated Events

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.



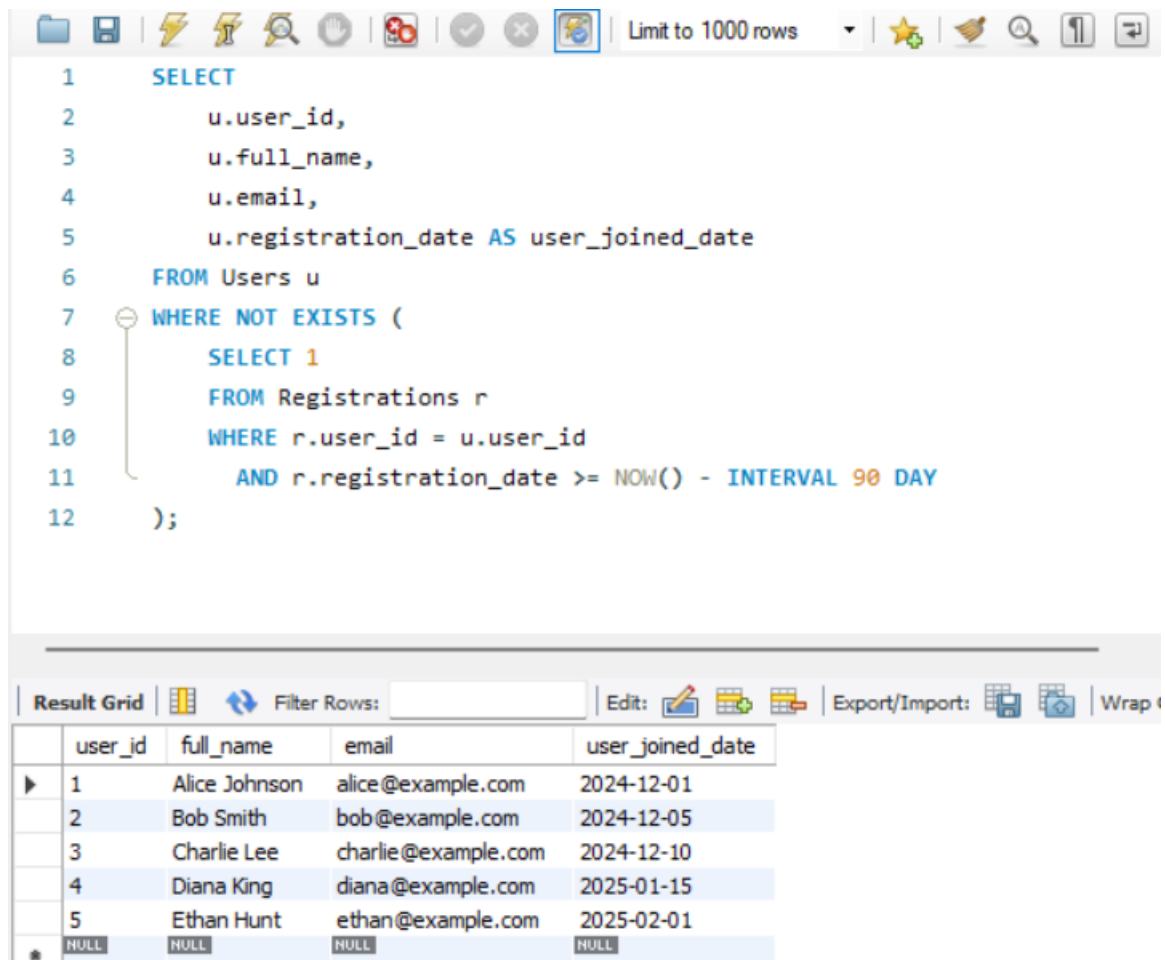
```
1 • SELECT
2     e.event_id,
3     e.title AS event_title,
4     AVG(f.rating) AS average_rating,
5     COUNT(f.feedback_id) AS total_feedback
6 FROM Events e
7 INNER JOIN Feedback f ON e.event_id = f.event_id
8 GROUP BY e.event_id, e.title
9 HAVING COUNT(f.feedback_id) >= 2
10 ORDER BY average_rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	event_id	event_title	average_rating	total_feedback
▶	2	AI & ML Conference	4.5000	2

3. Inactive Users

Retrieve users who have not registered for any events in the last 90 days.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

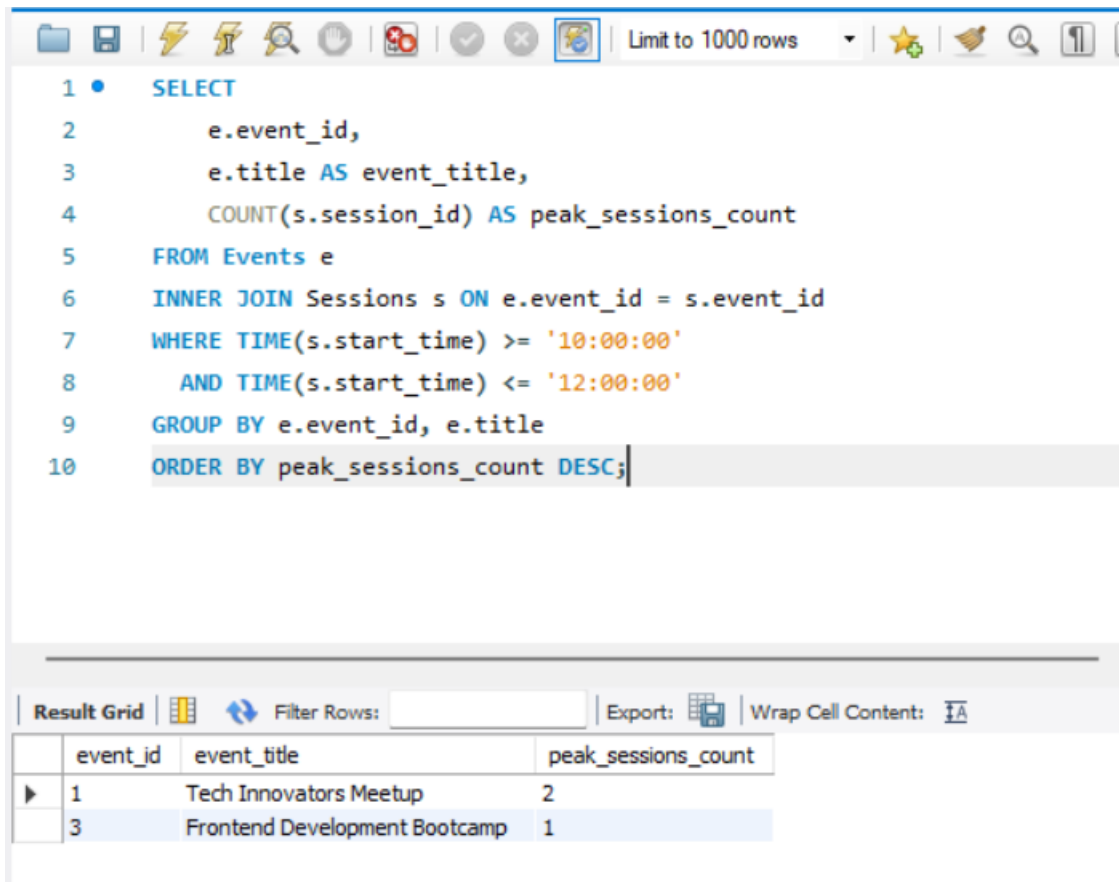
```
1  SELECT
2      u.user_id,
3      u.full_name,
4      u.email,
5      u.registration_date AS user_joined_date
6  FROM Users u
7  WHERE NOT EXISTS (
8      SELECT 1
9      FROM Registrations r
10     WHERE r.user_id = u.user_id
11           AND r.registration_date >= NOW() - INTERVAL 90 DAY
12 );
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input and buttons for 'Edit', 'Export/Import', and 'Wrap'. The results are displayed in a table with the following data:

	user_id	full_name	email	user_joined_date
▶	1	Alice Johnson	alice@example.com	2024-12-01
	2	Bob Smith	bob@example.com	2024-12-05
	3	Charlie Lee	charlie@example.com	2024-12-10
	4	Diana King	diana@example.com	2025-01-15
	5	Ethan Hunt	ethan@example.com	2025-02-01
•	NULL	NULL	NULL	NULL

4. Peak Session Hours

Count how many sessions are scheduled between 10 AM to 12 PM for each event.



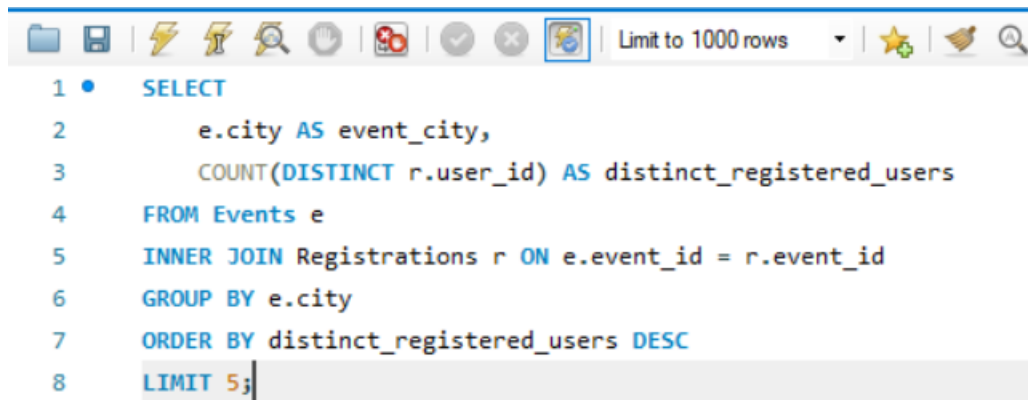
```
1 • SELECT
2     e.event_id,
3     e.title AS event_title,
4     COUNT(s.session_id) AS peak_sessions_count
5 FROM Events e
6 INNER JOIN Sessions s ON e.event_id = s.event_id
7 WHERE TIME(s.start_time) >= '10:00:00'
8     AND TIME(s.start_time) <= '12:00:00'
9 GROUP BY e.event_id, e.title
10 ORDER BY peak_sessions_count DESC;
```

Result Grid

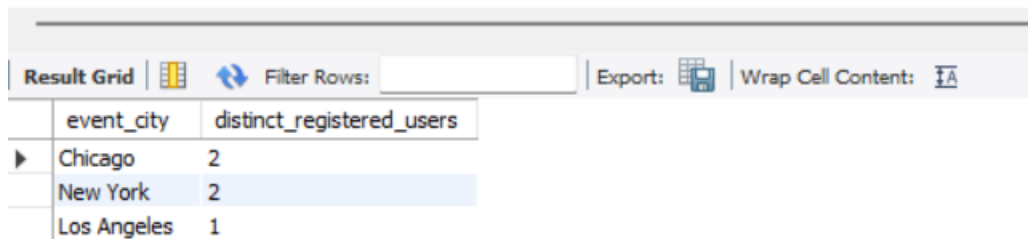
	event_id	event_title	peak_sessions_count
▶	1	Tech Innovators Meetup	2
	3	Frontend Development Bootcamp	1

5. Most Active Cities

List the top 5 cities with the highest number of distinct user registrations.



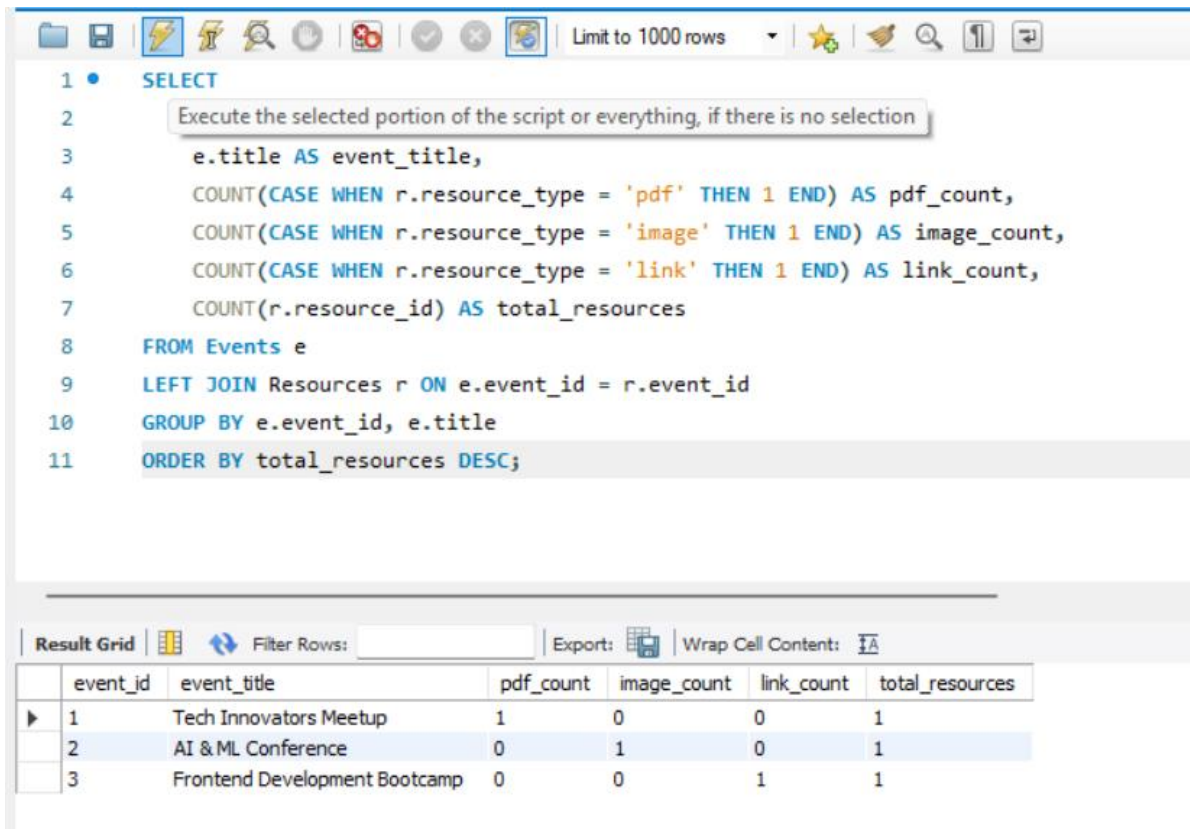
```
1 • SELECT
2     e.city AS event_city,
3     COUNT(DISTINCT r.user_id) AS distinct_registered_users
4 FROM Events e
5 INNER JOIN Registrations r ON e.event_id = r.event_id
6 GROUP BY e.city
7 ORDER BY distinct_registered_users DESC
8 LIMIT 5;
```



event_city	distinct_registered_users
Chicago	2
New York	2
Los Angeles	1

6. Event Resource Summary

Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
1 • SELECT
2     Execute the selected portion of the script or everything, if there is no selection
3     e.title AS event_title,
4     COUNT(CASE WHEN r.resource_type = 'pdf' THEN 1 END) AS pdf_count,
5     COUNT(CASE WHEN r.resource_type = 'image' THEN 1 END) AS image_count,
6     COUNT(CASE WHEN r.resource_type = 'link' THEN 1 END) AS link_count,
7     COUNT(r.resource_id) AS total_resources
8 FROM Events e
9 LEFT JOIN Resources r ON e.event_id = r.event_id
10 GROUP BY e.event_id, e.title
11 ORDER BY total_resources DESC;
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input and an 'Export' button. The results are displayed in a table with 7 columns: event_id, event_title, pdf_count, image_count, link_count, and total_resources.

event_id	event_title	pdf_count	image_count	link_count	total_resources
1	Tech Innovators Meetup	1	0	0	1
2	AI & ML Conference	0	1	0	1
3	Frontend Development Bootcamp	0	0	1	1

7. Low Feedback Alerts

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

The screenshot shows a SQL query editor with a toolbar at the top containing icons for file operations, execution, and search. The query text is as follows:

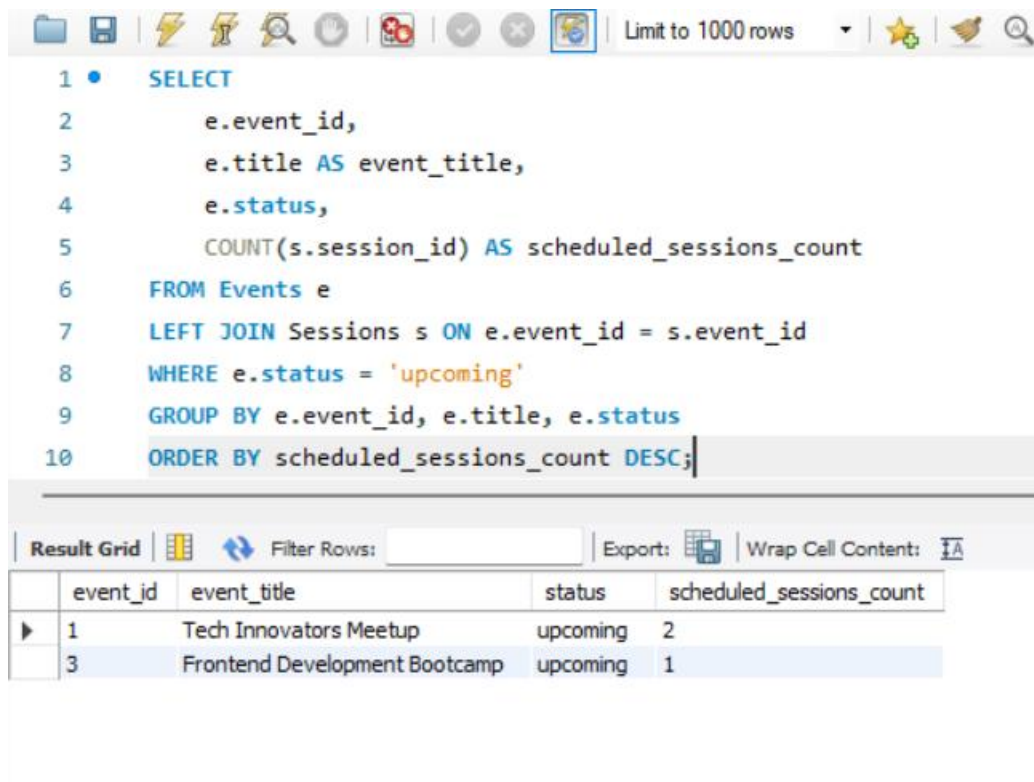
```
1 • SELECT
2     u.user_id,
3     u.full_name AS user_name,
4     e.title AS event_title,
5     f.rating,
6     f.comments,
7     f.feedback_date
8 FROM Feedback f
9 INNER JOIN Users u ON f.user_id = u.user_id
10 INNER JOIN Events e ON f.event_id = e.event_id
11 WHERE f.rating < 3
12 ORDER BY f.rating ASC;
```

Below the query editor, there is a toolbar with options for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. A table header is displayed below the toolbar:

	user_id	user_name	event_title	rating	comments	feedback_date
--	---------	-----------	-------------	--------	----------	---------------

8. Sessions per Upcoming Event

Display all upcoming events with the count of sessions scheduled for them.



The screenshot displays a SQL query editor interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL query is as follows:

```
1 • SELECT
2     e.event_id,
3     e.title AS event_title,
4     e.status,
5     COUNT(s.session_id) AS scheduled_sessions_count
6 FROM Events e
7 LEFT JOIN Sessions s ON e.event_id = s.event_id
8 WHERE e.status = 'upcoming'
9 GROUP BY e.event_id, e.title, e.status
10 ORDER BY scheduled_sessions_count DESC;
```

Below the query editor is the 'Result Grid' section, which includes a 'Filter Rows' input and an 'Export' button. The result grid contains the following data:

	event_id	event_title	status	scheduled_sessions_count
▶	1	Tech Innovators Meetup	upcoming	2
	3	Frontend Development Bootcamp	upcoming	1

9. Organizer Event Summary

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

Limit to 1000 rows

```

1 • SELECT
2     u.user_id AS organizer_id,
3     u.full_name AS organizer_name,
4     e.status AS event_status,
5     COUNT(e.event_id) AS total_events_created
6 FROM Users u
7 INNER JOIN Events e ON u.user_id = e.organizer_id
8 GROUP BY u.user_id, u.full_name, e.status
9 ORDER BY organizer_name ASC, total_events_created DESC;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	organizer_id	organizer_name	event_status	total_events_created
▶	1	Alice Johnson	upcoming	1
	2	Bob Smith	upcoming	1
	3	Charlie Lee	completed	1

10. Feedback Gap

Identify events that had registrations but received no feedback at all.

Limit to 1000 rows

```

1 • SELECT
2     e.event_id,
3     e.title AS event_title
4 FROM Events e
5 INNER JOIN Registrations r ON e.event_id = r.event_id
6 LEFT JOIN Feedback f ON e.event_id = f.event_id
7 WHERE f.feedback_id IS NULL
8 GROUP BY e.event_id, e.title;

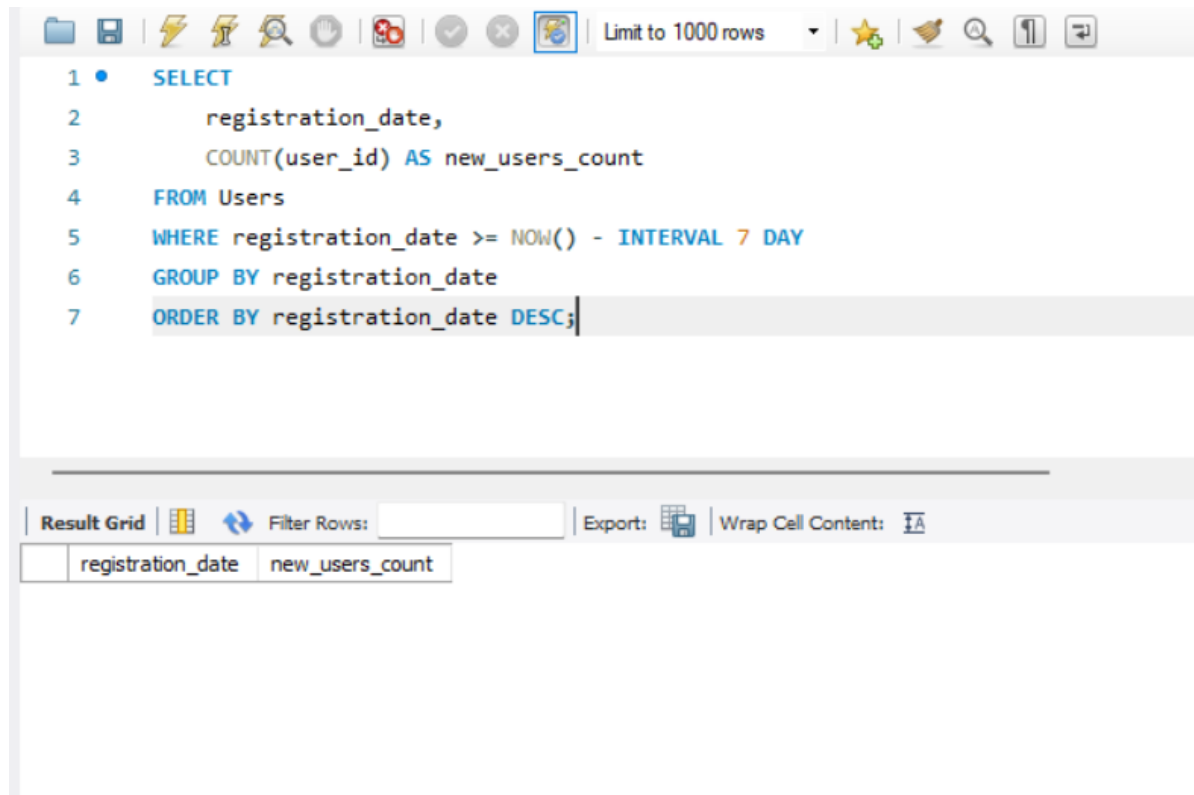
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	event_id	event_title
▶	3	Frontend Development Bootcamp

11. Daily New User Count

Find the number of users who registered each day in the last 7 days.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
1 • SELECT
2     registration_date,
3     COUNT(user_id) AS new_users_count
4 FROM Users
5 WHERE registration_date >= NOW() - INTERVAL 7 DAY
6 GROUP BY registration_date
7 ORDER BY registration_date DESC;
```

Below the editor is a 'Result Grid' section with a 'Filter Rows' input and an 'Export' button. The grid header shows two columns: 'registration_date' and 'new_users_count'.

12. Event with Maximum Sessions

List the event(s) with the highest number of sessions.

SQL Query Editor interface showing a query to find events with the most sessions.

```

5 FROM Events e
6 INNER JOIN Sessions s ON e.event_id = s.event_id
7 GROUP BY e.event_id, e.title
8 HAVING COUNT(s.session_id) = (
9     SELECT COUNT(session_id)
10    FROM Sessions
11   GROUP BY event_id
12  ORDER BY COUNT(session_id) DESC
13  LIMIT 1
14 );

```

Result Grid:

event_id	event_title	total_sessions
1	Tech Innovators Meetup	2

13. Average Rating per City

Calculate the average feedback rating of events conducted in each city.

SQL Query Editor interface showing a query to calculate the average feedback rating per city.

```

1 • SELECT
2     e.city AS event_city,
3     ROUND(AVG(f.rating), 2) AS average_feedback_rating,
4     COUNT(f.feedback_id) AS total_reviews_received
5 FROM Events e
6 INNER JOIN Feedback f ON e.event_id = f.event_id
7 GROUP BY e.city
8 ORDER BY average_feedback_rating DESC;

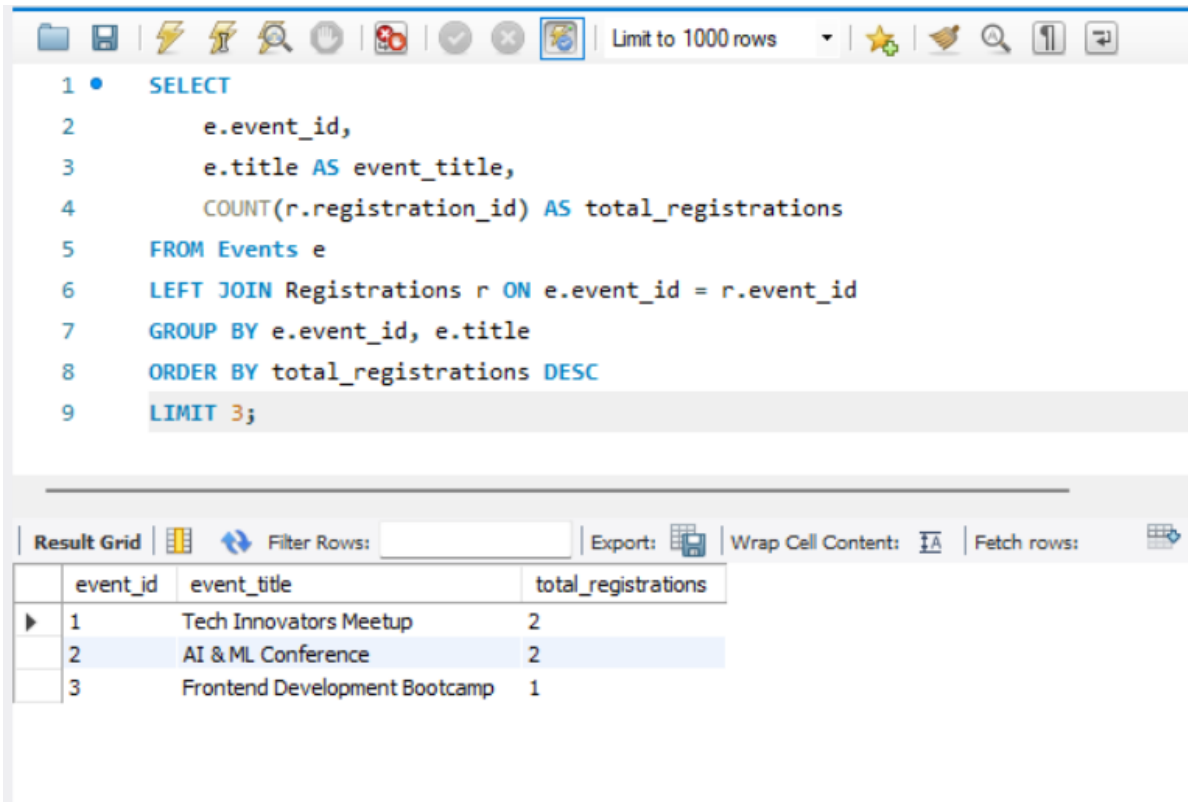
```

Result Grid:

event_city	average_feedback_rating	total_reviews_received
Chicago	4.50	2
New York	3.00	1

14. Most Registered Events

List top 3 events based on the total number of user registrations.



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

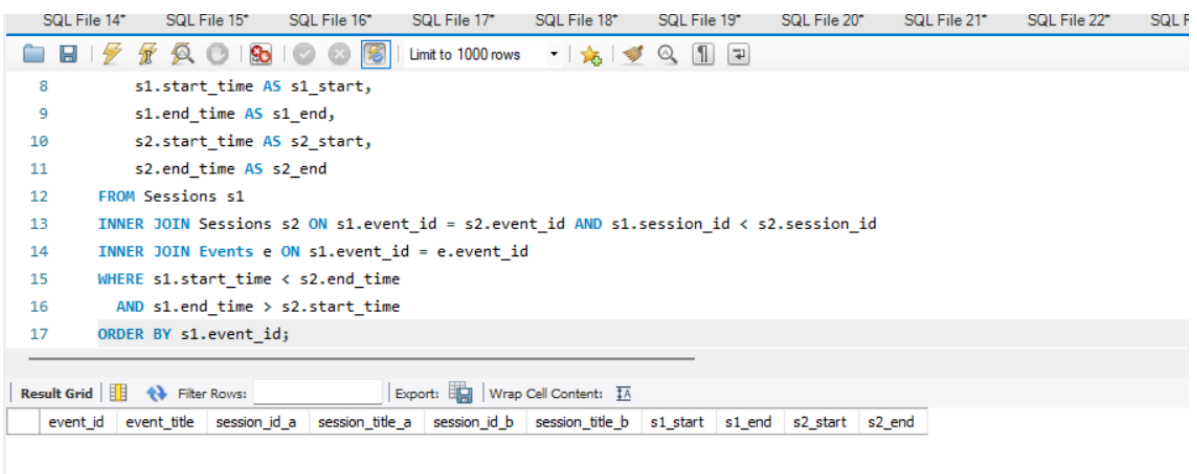
```
1 SELECT
2     e.event_id,
3     e.title AS event_title,
4     COUNT(r.registration_id) AS total_registrations
5 FROM Events e
6 LEFT JOIN Registrations r ON e.event_id = r.event_id
7 GROUP BY e.event_id, e.title
8 ORDER BY total_registrations DESC
9 LIMIT 3;
```

The result grid displays the following data:

	event_id	event_title	total_registrations
1	1	Tech Innovators Meetup	2
2	2	AI & ML Conference	2
3	3	Frontend Development Bootcamp	1

15. Event Session Time Conflict

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
8 s1.start_time AS s1_start,
9 s1.end_time AS s1_end,
10 s2.start_time AS s2_start,
11 s2.end_time AS s2_end
12 FROM Sessions s1
13 INNER JOIN Sessions s2 ON s1.event_id = s2.event_id AND s1.session_id < s2.session_id
14 INNER JOIN Events e ON s1.event_id = e.event_id
15 WHERE s1.start_time < s2.end_time
16 AND s1.end_time > s2.start_time
17 ORDER BY s1.event_id;
```

The result grid displays the following data:

event_id	event_title	session_id_a	session_title_a	session_id_b	session_title_b	s1_start	s1_end	s2_start	s2_end
----------	-------------	--------------	-----------------	--------------	-----------------	----------	--------	----------	--------

16. Unregistered Active Users

Find users who created an account in the last 30 days but haven't registered for any events.

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```

1 • SELECT
2     u.user_id,
3     u.full_name,
4     u.email,
5     u.registration_date AS account_created_date
6 FROM Users u
7 LEFT JOIN Registrations r ON u.user_id = r.user_id
8 WHERE u.registration_date >= NOW() - INTERVAL 30 DAY
9     AND r.registration_id IS NULL;

```

Below the query editor, there is a toolbar with options like "Result Grid", "Filter Rows", "Export", and "Wrap Cell Content". A table header is visible below the toolbar:

user_id	full_name	email	account_created_date
---------	-----------	-------	----------------------

17. Multi-Session Speakers

Identify speakers who are handling more than one session across all events.

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```

1 • SELECT
2     speaker_name,
3     COUNT(session_id) AS total_sessions_handled,
4     COUNT(DISTINCT event_id) AS distinct_events_count
5 FROM Sessions
6 GROUP BY speaker_name
7 HAVING COUNT(session_id) > 1
8 ORDER BY total_sessions_handled DESC;

```

Below the query editor, there is a toolbar with options like "Result Grid", "Filter Rows", "Export", and "Wrap Cell Content". A table header is visible below the toolbar:

speaker_name	total_sessions_handled	distinct_events_count
--------------	------------------------	-----------------------

18. Resource Availability Check

List all events that do not have any resources uploaded.

SQL Query Editor interface showing a query to find events with no resources. The query is as follows:

```

1 • SELECT
2     e.event_id,
3     e.title AS event_title,
4     e.city
5 FROM Events e
6 LEFT JOIN Resources r ON e.event_id = r.event_id
7 WHERE r.resource_id IS NULL;

```

The interface includes a toolbar with icons for file operations, a "Limit to 1000 rows" dropdown, and a "Result Grid" section at the bottom. The "Result Grid" section shows the columns: event_id, event_title, and city.

19. Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

SQL Query Editor interface showing a query to find completed events with feedback summary. The query is as follows:

```

8     SELECT event_id, COUNT(registration_id) AS total_registrations
9     FROM Registrations
10    GROUP BY event_id
11 ) reg ON e.event_id = reg.event_id
12 LEFT JOIN (
13     SELECT event_id, AVG(rating) AS avg_rating
14     FROM Feedback
15    GROUP BY event_id
16 ) feed ON e.event_id = feed.event_id
17 WHERE e.status = 'completed';

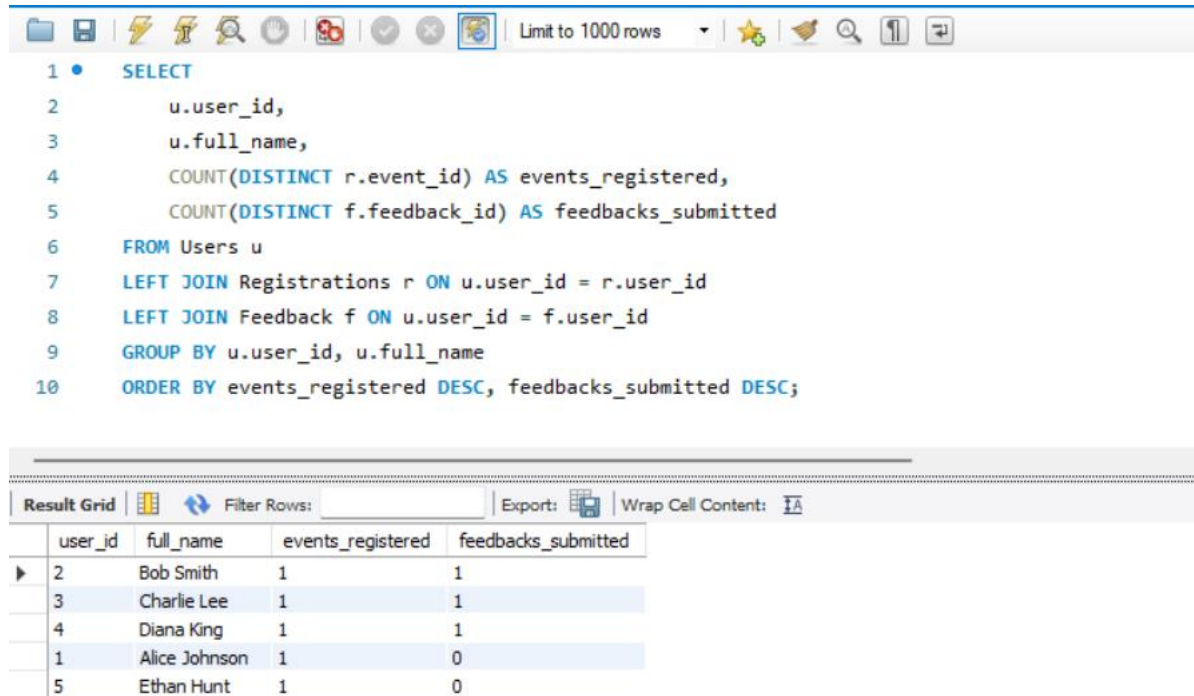
```

The interface includes a toolbar with icons for file operations, a "Limit to 1000 rows" dropdown, and a "Result Grid" section at the bottom. The "Result Grid" section shows the columns: event_id, event_title, total_registrations, and average_rating. The data row shows:

event_id	event_title	total_registrations	average_rating
2	AI & ML Conference	2	4.50

20. User Engagement Index

For each user, calculate how many events they attended and how many feedbacks they submitted.



The screenshot shows a SQL query editor interface. The query is as follows:

```
1 • SELECT
2     u.user_id,
3     u.full_name,
4     COUNT(DISTINCT r.event_id) AS events_registered,
5     COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted
6 FROM Users u
7 LEFT JOIN Registrations r ON u.user_id = r.user_id
8 LEFT JOIN Feedback f ON u.user_id = f.user_id
9 GROUP BY u.user_id, u.full_name
10 ORDER BY events_registered DESC, feedbacks_submitted DESC;
```

Below the query editor is the 'Result Grid' showing the output of the query. The grid has columns for user_id, full_name, events_registered, and feedbacks_submitted. The results are sorted by events_registered in descending order, and then by feedbacks_submitted in descending order.

	user_id	full_name	events_registered	feedbacks_submitted
▶	2	Bob Smith	1	1
	3	Charlie Lee	1	1
	4	Diana King	1	1
	1	Alice Johnson	1	0
	5	Ethan Hunt	1	0

21. Top Feedback Providers

List top 5 users who have submitted the most feedback entries.

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with various icons and a dropdown menu set to "Limit to 1000 rows". Below the toolbar, a SQL query is entered in a text area:

```

1 • SELECT
2     u.user_id,
3     u.full_name,
4     u.email,
5     COUNT(f.feedback_id) AS feedback_count
6 FROM Users u
7 INNER JOIN Feedback f ON u.user_id = f.user_id
8 GROUP BY u.user_id, u.full_name, u.email
9 ORDER BY feedback_count DESC
10 LIMIT 5;

```

Below the query editor, there is a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	user_id	full_name	email	feedback_count
▶	2	Bob Smith	bob@example.com	1
	3	Charlie Lee	charlie@example.com	1
	4	Diana King	diana@example.com	1

22. Duplicate Registrations Check

Detect if a user has been registered more than once for the same event.

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with various icons and a dropdown menu set to "Limit to 1000 rows". Below the toolbar, a SQL query is entered in a text area:

```

1 • SELECT
2     user_id,
3     event_id,
4     COUNT(registration_id) AS registration_count
5 FROM Registrations
6 GROUP BY user_id, event_id
7 HAVING COUNT(registration_id) > 1;

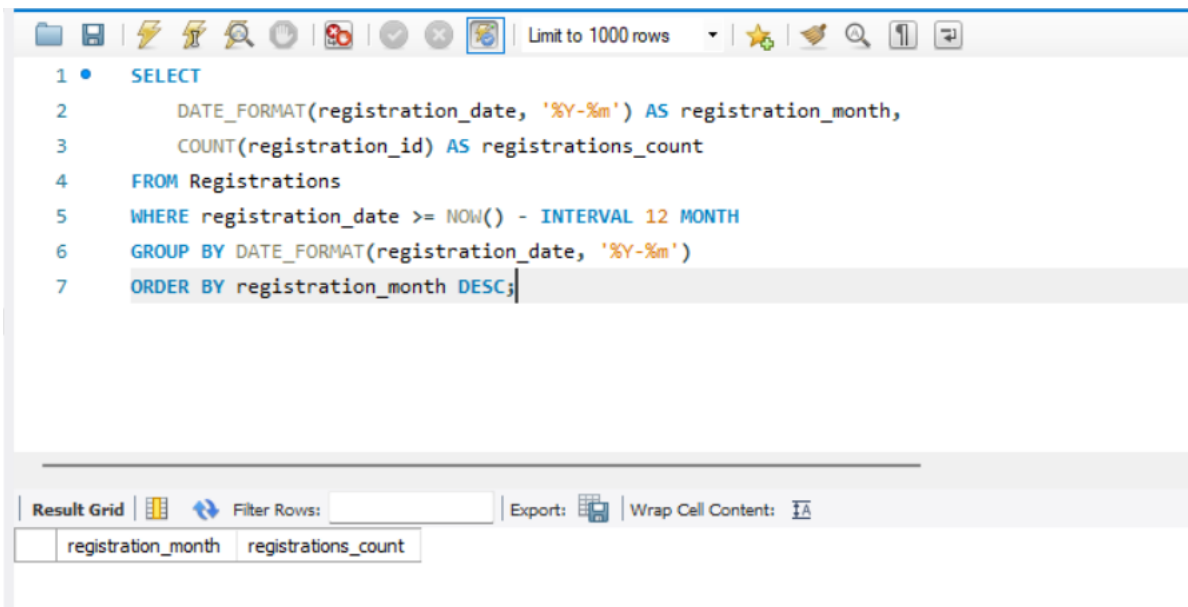
```

Below the query editor, there is a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	user_id	event_id	registration_count
--	---------	----------	--------------------

23. Registration Trends

Show a month-wise registration count trend over the past 12 months.



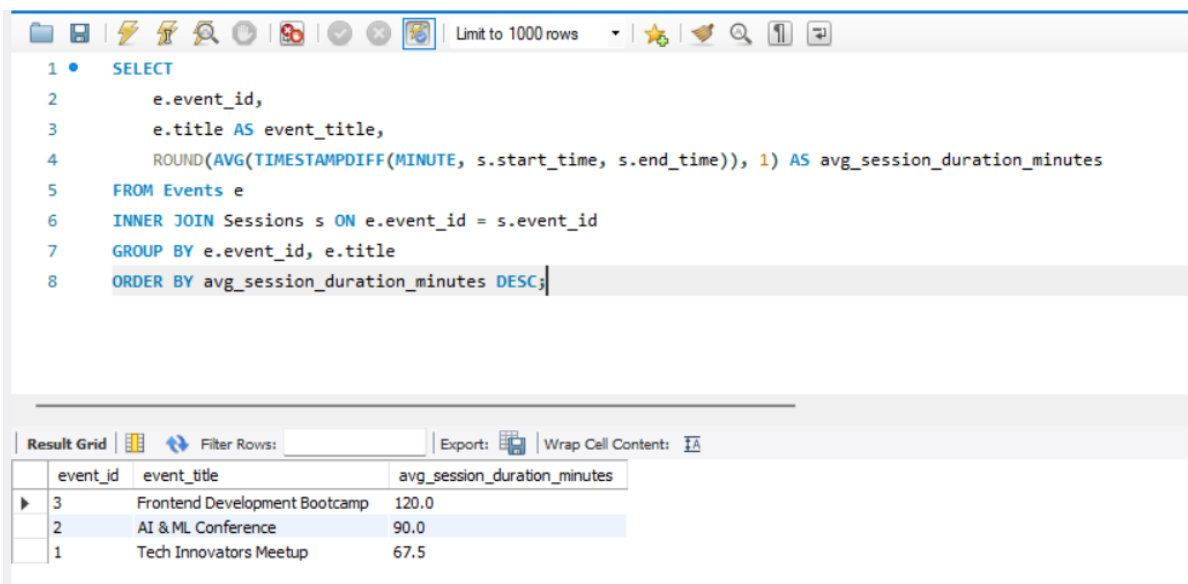
The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • SELECT
2     DATE_FORMAT(registration_date, '%Y-%m') AS registration_month,
3     COUNT(registration_id) AS registrations_count
4 FROM Registrations
5 WHERE registration_date >= NOW() - INTERVAL 12 MONTH
6 GROUP BY DATE_FORMAT(registration_date, '%Y-%m')
7 ORDER BY registration_month DESC;
```

Below the query editor, the 'Result Grid' tab is active, showing a table with two columns: 'registration_month' and 'registrations_count'.

24. Average Session Duration per Event

Compute the average duration (in minutes) of sessions in each event.



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 • SELECT
2     e.event_id,
3     e.title AS event_title,
4     ROUND(AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time)), 1) AS avg_session_duration_minutes
5 FROM Events e
6 INNER JOIN Sessions s ON e.event_id = s.event_id
7 GROUP BY e.event_id, e.title
8 ORDER BY avg_session_duration_minutes DESC;
```

Below the query editor, the 'Result Grid' tab is active, showing a table with three columns: 'event_id', 'event_title', and 'avg_session_duration_minutes'. The table contains three rows of data:

event_id	event_title	avg_session_duration_minutes
3	Frontend Development Bootcamp	120.0
2	AI & ML Conference	90.0
1	Tech Innovators Meetup	67.5

25. Events Without Sessions

List all events that currently have no sessions scheduled under them.

